

A PEDAGOGICAL PAPER OF THE SERIES OF AUTONOMOUS SYSTEMS IN FINANCE

UNDERSTANDING THE VAULT-RAG NOTEBOOK

A Simple Case of Use for Knowledge Vaults

“Index once, retrieve many times, and generate on demand.”

Alejandro Reynoso

JUDGE BUSINESS SCHOOL, UNIVERSITY OF CAMBRIDGE. JULY 2026

Abstract

This paper explains a notebook that turns an Obsidian knowledge vault into a simple Retrieval-Augmented Generation (RAG) system. The central purpose is not merely to describe a programming workflow, but to make the logic of the workflow understandable to researchers who may write extensively in Markdown while having little prior experience with vector search, embeddings, or language-model grounding. The notebook follows a practical sequence: it brings vault files into a cloud notebook environment, reads and cleans Markdown notes, divides long documents into overlapping chunks, converts those chunks into semantic vectors, stores the vectors in a FAISS index, retrieves the chunks most relevant to a user question, and finally gives those chunks to a generative model so that it can draft grounded prose. In this sense, the vault becomes more than an archive. It becomes an active research partner that can surface old ideas, connect scattered notes, and support the drafting of larger arguments. The paper emphasizes why each stage exists, what problem each stage solves, and what design tradeoffs a researcher must understand when adapting the pattern to a new project. It also treats the notebook as a teaching artifact: a small but complete example of how personal knowledge management, information retrieval, and generative artificial intelligence can be combined in a controlled workflow.

Keywords: Retrieval-Augmented Generation; Obsidian; knowledge vault; semantic search; embeddings; FAISS; research writing; grounded generation.

Contents

1	Why This Notebook Exists	3
2	The Core Idea, Stripped to Its Essentials	3
3	Stage 0: Getting Your Files Into Colab	4
4	Stage 1: Turning Notes Into Chunks	5
5	Stage 1, Continued: From Text to Numbers	6
6	Stage 2: Retrieval, in Practice	7
7	Stage 3: Generation, With Grounding	8
8	A Simple Case of Use, Concretely	10
9	Research and Writing Implications	11
10	Closing Thought	11

1 Why This Notebook Exists

Researchers often accumulate far more written material than they can actively remember. A personal vault may begin as a modest collection of reading notes, project sketches, quotations, diagrams, meeting records, and unfinished reflections. Over time, however, that collection can become a private archive containing hundreds or thousands of Markdown files. The problem is not that the archive lacks value; the problem is that its value becomes difficult to reach at the moment when it is needed. A researcher may remember that they once wrote something about implementation, scientific discovery, method design, or the connection between two theoretical traditions, but not remember the exact filename, date, wording, or folder where the idea was stored. Traditional search helps when the researcher knows the precise word to search for. It is much weaker when the researcher remembers a meaning, a problem, or a relationship rather than an exact phrase.

The notebook exists to address that gap. It shows how an Obsidian vault can be transformed from a passive collection of documents into a searchable semantic memory. The key idea is Retrieval-Augmented Generation, usually shortened to RAG. In a RAG workflow, a language model does not answer only from patterns learned during training. Instead, the system first retrieves relevant material from a specific knowledge source, then asks the model to write using that material as context [1]. The retrieval step matters because it gives the generated answer an anchor outside the model itself. For a researcher, this means the output can be tied to notes that already exist in the vault, rather than floating freely as plausible but unsupported prose.

This distinction is especially important in academic writing. Researchers do not simply want fluent paragraphs; they want paragraphs that are traceable to evidence, prior reasoning, and documented sources. A generic language model can produce an impressive explanation of almost any topic, but it may also invent details, blur distinctions, or supply arguments that do not reflect the researcher's actual materials. A vault-based RAG notebook changes the task. Instead of asking, "What can the model say about this topic?" the researcher asks, "What can be written from the material I have already collected?" That shift makes the workflow more transparent and more responsible.

The notebook also exists as a bridge between two familiar research practices. On one side is personal knowledge management, where tools such as Obsidian help writers build networks of notes, tags, links, and ideas. On the other side is computational retrieval, where documents are represented numerically so that a computer can compare them by similarity. Many researchers understand the first practice but not the second. The notebook makes the second practice concrete by showing every stage in code. It is not a production system with elaborate infrastructure; it is a teaching example that can run in a notebook environment and make the hidden mechanics visible.

Finally, the notebook exists because the simplest useful version of RAG is already powerful. It does not require a custom database server, a complex web application, or a team of engineers. The essential pattern is modest: read files, split text, embed chunks, search vectors, and send retrieved context to a model. Once that pattern is understood, the researcher can adapt it to many writing tasks, including literature reviews, grant proposals, theoretical essays, project summaries, and article sections. The notebook therefore functions as both a working tool and a conceptual map. It helps the user do something immediately useful while also teaching a reusable structure for future knowledge-work systems.

2 The Core Idea, Stripped to Its Essentials

The whole pipeline can be summarized in one sentence: turn notes into searchable numbers, turn a question into the same kind of numbers, find the closest notes, and ask a language model to write with those notes in view. This sentence is simple, but it contains the major conceptual leap

behind modern semantic retrieval. A computer does not naturally understand that “academic implementation,” “putting a theory into practice,” and “operationalizing a research idea” may refer to related intellectual tasks. A keyword search can miss that relationship because the words differ. An embedding model, by contrast, maps pieces of text into vectors that are designed to place semantically similar passages near one another in a mathematical space [3]. The notebook uses this property to make a personal vault searchable by meaning rather than only by literal wording.

The first stage is indexing. Indexing means preparing the vault so that it can be searched efficiently later. The notebook reads each Markdown note, removes metadata that should not dominate the search, divides long notes into smaller chunks, computes an embedding vector for every chunk, and stores those vectors in a FAISS index. This stage is usually done before asking substantive questions. It is similar to building a library catalog: the work may take some time, but once it is done, many searches become fast and organized. Without indexing, every question would require rereading and recomputing the entire vault from the beginning.

The second stage is retrieval. Retrieval begins when the user asks a question or gives a writing instruction. The notebook embeds the question with the same embedding model used for the notes. It then compares the question vector with the stored note vectors and returns the chunks that are most similar. This is where the system decides which parts of the vault should be placed in front of the language model. Retrieval is therefore a filtering operation. It narrows a large archive to a small, relevant set of passages that fit inside the model’s context window. If retrieval works well, the generation stage receives enough evidence to write usefully without being overwhelmed by irrelevant material.

The third stage is generation. Generation is the part that looks most familiar to users of AI writing tools: a prompt is sent to a language model, and the model produces an answer or draft. The difference is that the prompt now includes retrieved material from the vault. The model is instructed to use that material, cite the originating note filenames, and acknowledge when the vault does not contain enough relevant information. These instructions are important because RAG is not automatically reliable. The system must be designed to encourage grounded writing rather than confident invention. In practice, the prompt serves as a contract: write from the supplied context, do not pretend that missing evidence exists, and make the relationship between answer and source visible.

The notebook’s value comes from the coordination of these three stages. Indexing alone produces a searchable structure but no prose. Retrieval alone finds relevant passages but does not synthesize them. Generation alone writes fluently but may not be grounded in the researcher’s archive. Together, the stages create a workflow in which the vault supplies evidence, the retrieval system supplies relevance, and the model supplies synthesis. This division of labor is what makes RAG attractive for research writing. It preserves the archive as the source of authority while using the model as a drafting and organizing assistant.

The stripped-down version is also useful because it prevents the technology from seeming magical. A RAG system is not a mysterious agent that understands an entire vault in a human way. It is a sequence of operations with limits. Chunk size affects what can be retrieved. Embedding quality affects which meanings are captured. The value of k , the number of returned chunks, affects how broad or narrow the context becomes. Prompt design affects whether the model cites sources or wanders beyond them. Understanding these moving parts makes the researcher a more capable user and critic of the system.

3 Stage 0: Getting Your Files Into Colab

Before the notebook can perform retrieval, it must be able to see the notes. This preliminary step is called Stage 0 because it is not part of the conceptual RAG triad of indexing, retrieval, and

generation, yet it is necessary for the workflow to function. Google Colab runs code in a remote runtime, not directly on the user's Chromebook or local computer. That means the notebook cannot automatically read files that exist only on the user's device. The notes must be placed somewhere the runtime can access. The notebook handles this through Google Drive, which acts as a bridge between the local writing environment and the cloud computation environment.

In practice, the user copies the Obsidian vault into Drive, opens the notebook in Colab, and mounts Drive from inside the notebook. Mounting makes Drive appear to the notebook like a normal folder. The code can then walk through the vault directory, find Markdown files, and load their contents. The example below names the mount point symbolically so that the manuscript does not depend on a machine-specific path.

```
from google.colab import drive

DRIVE_MOUNT = "content/drive"
drive.mount(DRIVE_MOUNT)

VAULT_DIR = "content/drive/MyDrive/ObsidianVault"
```

This simple step teaches an important lesson about cloud notebooks: computation and data must be brought into the same environment. The notebook may contain all the correct code, but it cannot index a vault that it cannot access. Many beginners experience errors at this point because they assume that a cloud notebook can see the same folders as their personal machine. It cannot. The runtime is a separate computer. The bridge may be Drive, a file upload panel, a cloud storage bucket, or a synchronized folder, but some bridge must exist.

The Drive approach is especially appropriate for a Chromebook workflow. Chromebooks are often used with cloud storage by default, and Colab is tightly integrated with Google accounts. This reduces the number of setup steps. The researcher does not need to configure a server, install a local Python environment, or manage operating-system-specific dependencies. They mainly need to ensure that the vault has been copied to a predictable location and that the notebook points to the correct folder. This design lowers the barrier for researchers who want to learn the logic of RAG without first becoming system administrators.

Stage 0 also raises a practical issue about file organization. A vault may contain notes, templates, images, attachments, plugin files, and configuration folders. The notebook should focus on the Markdown notes that contain prose, not every artifact in the vault. For that reason, the file-loading code typically filters for filenames ending in `.md`. Depending on the vault, additional filtering may be useful. A researcher may want to exclude daily notes, private journal entries, draft fragments, or folders that contain administrative material. These choices affect the character of the system because retrieval can only draw from what was indexed. If irrelevant or sensitive notes are included, they may appear later in generated context.

The most important point is that Stage 0 is not merely technical housekeeping. It defines the boundary of the knowledge source. Whatever is placed in the vault directory becomes potential evidence for later generation. Whatever is excluded becomes invisible to the RAG system. A careful researcher should therefore treat this stage as a moment of curation. Before indexing, it is worth asking what kind of writing task the system is meant to support and which notes should legitimately inform that task. Good retrieval begins before the first embedding is computed; it begins with a thoughtfully prepared corpus.

4 Stage 1: Turning Notes Into Chunks

Once the notebook can see the vault, it must transform the notes into units that can be searched effectively. Obsidian notes are Markdown files, which usually combine plain prose with headings, links, tags, lists, and sometimes a metadata block at the top called frontmatter. Frontmatter

may record dates, aliases, tags, project labels, or other structured information. That information can be useful for organization, but it is not always useful as the main semantic content of the note. The notebook therefore loads each Markdown file and separates the content from the metadata. The goal is to retrieve the ideas in the note rather than letting tags or administrative labels dominate the comparison.

```
post = frontmatter.load(path)
text = post.content
```

After loading the text, the notebook divides it into chunks. Chunking is necessary because a note can be too long and too internally varied to serve as a single retrieval unit. A long note might begin with a definition, move into a literature summary, record a personal critique, and end with a project idea. If the entire note is represented by one vector, the vector becomes an average of all those parts. That average may be too broad to match a precise question. By splitting the note into smaller passages, the notebook allows retrieval to find the specific region of the note that matters.

The notebook uses chunks of about 800 words with an overlap of about 150 words. These numbers are not universal laws; they are practical starting points. A chunk should be large enough to contain context, argument, and meaning, but small enough to remain focused. If chunks are too short, the model may retrieve fragments that lack enough surrounding explanation to be useful. If chunks are too long, the retrieval system may return broad passages with too many unrelated ideas. The overlap reduces boundary problems. If a key explanation begins at the end of one chunk and continues into the next, overlap ensures that at least one retrieved chunk may contain the complete thought.

```
def chunk_text(text, chunk_size=800, overlap=150):
    words = text.split()
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start = end - overlap
    return chunks
```

Chunking also creates a useful link between retrieval and citation. Each chunk can be stored with metadata indicating the source filename and the chunk number. Later, when the language model receives retrieved passages, the prompt can show where each passage came from. This lets the model cite filenames and lets the researcher inspect the original notes. Without chunk metadata, the system might produce a useful answer but leave the user unable to verify the basis for that answer. For research writing, verification is not optional; it is part of the method.

There is also a conceptual reason to emphasize chunking. It reminds us that a RAG system does not retrieve whole books of knowledge in a human way. It retrieves prepared segments. The quality of those segments shapes the quality of the final answer. If the notes are messy, if headings are unclear, if many unrelated ideas are packed into one file, or if the chunking strategy cuts arguments in awkward places, retrieval may suffer. Researchers can improve the system not only by changing code but also by improving their note-taking habits. Clear headings, focused notes, and consistent summaries all make the vault more retrievable. In this sense, the notebook encourages a feedback loop between writing practice and computational search.

5 Stage 1, Continued: From Text to Numbers

After chunking, the notebook must convert each text chunk into a form that a computer can compare efficiently. This is the role of embeddings. An embedding is a vector, or ordered list of

numbers, produced by a model trained to place semantically related texts near one another in vector space. When two chunks express similar meanings, their vectors tend to point in similar directions. When they discuss unrelated topics, their vectors tend to be farther apart. This numerical representation is what allows the notebook to search by meaning rather than by exact word matching.

The notebook uses `all-MiniLM-L6-v2`, a compact sentence-transformer model. It is small enough to run conveniently in a notebook environment, yet strong enough for many semantic search tasks [3]. The model reads each chunk and returns a vector. The notebook normalizes the vectors so that similarity can be computed using inner product in a way that corresponds closely to cosine similarity. Normalization is a practical detail, but it matters because vector comparison depends on both direction and magnitude unless the vectors are scaled consistently.

```
embed_model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = embed_model.encode(chunks, normalize_embeddings=True)
```

The privacy implications of this design are worth noting. In the indexing stage, the notes are processed by a model running inside the notebook environment. No external generative API is needed just to compute embeddings. This does not mean the entire workflow is private in every possible sense, because the notebook still runs in a cloud environment and later sends retrieved chunks to a generation service. But it does mean that the full vault does not need to be transmitted to a language-model provider for indexing. Only the selected passages used for generation are passed forward at the final stage. For researchers working with sensitive notes, this distinction is significant. It supports a more selective exposure model: retrieve locally first, then share only the minimum context needed for the requested writing task.

After the embeddings are computed, the notebook stores them in a FAISS index. FAISS, short for Facebook AI Similarity Search, is a library designed for efficient vector similarity search over large collections [2]. In the notebook's simple version, the index is flat, meaning it compares the query vector against all stored vectors. This is easy to understand and appropriate for modest vault sizes. Larger collections may require more advanced index types that trade exactness for speed, but the flat index is pedagogically useful because it keeps the logic transparent.

```
index = faiss.IndexFlatIP(embeddings.shape[1])
index.add(embeddings)
```

The phrase “from text to numbers” should not make the process seem reductive in a negative sense. The notebook is not claiming that a vector captures everything meaningful about a passage. It cannot preserve every nuance, rhetorical gesture, or scholarly implication. Instead, the vector is a useful approximation for a specific task: finding passages that are likely to be relevant to a query. The researcher should understand embeddings as search instruments, not as complete interpretations. They help locate material; the researcher still evaluates, revises, and interprets the final result. This human oversight is essential because semantic similarity is not the same as scholarly adequacy, and because a technically close match may still require careful reading before it can become evidence in an argument.

6 Stage 2: Retrieval, in Practice

Retrieval begins when the researcher asks a question or gives an instruction. At that point, the notebook takes the user's query and embeds it using the same model that embedded the note chunks. This symmetry is crucial. If notes and queries were embedded by incompatible models, their vectors would not live in the same space and similarity scores would be meaningless. By using the same embedding model, the notebook makes the query comparable to every indexed chunk. The FAISS index then returns the closest vectors, along with similarity scores and identifiers that connect the vectors back to their source chunks.


```
def retrieve(query, k=8):
    q_emb = embed_model.encode([query], normalize_embeddings=True)
    scores, idxs = index.search(q_emb, k)
    return [(scores[0][i], metadata[idx]) for i, idx in enumerate(idxs[0])]
```

The parameter *k* controls how many chunks are retrieved. This value is more than a technical setting; it shapes the writing situation. A small *k*, such as three or five, gives the language model a narrow set of passages. This can improve focus and reduce noise, but it may miss useful context. A larger *k*, such as eight, twelve, or more, increases coverage. It gives the model more material to synthesize, but it also increases the chance that irrelevant or weakly related passages will enter the prompt. The best setting depends on the task. A precise factual question may benefit from a small *k*. A broad literature-review prompt may need a larger one.

Retrieval quality also depends on the wording of the query. Because embeddings capture semantic patterns, the query does not need to match the notes exactly, but it still needs to express the user's intention clearly. A vague query such as "science" may retrieve general material without a clear purpose. A more focused query such as "how scientific discovery becomes implemented in institutional practice" gives the system a richer semantic target. In this way, writing a query is itself a research skill. The researcher is not simply pressing a search button; they are framing a question in a way that guides the system toward useful evidence.

The notebook's retrieval output should be inspected, not blindly trusted. Good RAG practice includes looking at the retrieved chunks before relying on generated prose. Scores can suggest relevance, but they do not guarantee that a passage is accurate, current, or appropriate for the argument. A passage may be semantically similar while still being conceptually opposed to the user's intended claim. Another passage may contain a useful keyword but come from a speculative note that should not be treated as evidence. Retrieval narrows the archive, but human judgment still decides whether the retrieved material should support the final text.

This stage is also where the vault can surprise the researcher. Because semantic retrieval is not limited to exact keywords, it can surface notes that the researcher did not remember connecting. A query about implementation may retrieve a note on laboratory practice, a reflection on organizational adoption, and a reading summary about epistemic communities. These connections may not be obvious from filenames or folder structure. The system therefore functions not only as a search tool but also as a discovery tool. It can reveal latent relationships across notes written at different times for different reasons.

In practice, retrieval is the hinge of the whole notebook. If retrieval fails, generation will be weak no matter how capable the language model is. If retrieval succeeds, even a modest generation step can produce useful, grounded prose. This is why the notebook separates retrieval from generation instead of hiding it. The separation lets the researcher test queries, inspect chunks, adjust *k*, refine the corpus, and understand why a particular answer was produced. Transparency at this stage is one of the main educational strengths of the workflow.

7 Stage 3: Generation, With Grounding

Generation is the stage in which retrieved vault material becomes written output. The notebook gathers the selected chunks into a context block, places that context inside a prompt, and asks a language model to draft a section, answer, or explanation. This is the part of the workflow that most visibly resembles ordinary AI-assisted writing, but the surrounding structure changes its meaning. The model is not being asked to produce an answer from general memory alone. It is being asked to write in relation to a supplied set of passages that came from the researcher's own notes.

```
prompt = f"""
Use the vault context to write the requested section.
```

```
If the context is insufficient, say what is missing.

VAULT CONTEXT:
{context_block}

Write only the section content in Markdown.
"""

response = client.messages.create(
    model="claude-sonnet-4-6",
    messages=[{"role": "user", "content": prompt}],
)
```

The prompt design matters because grounding is not automatic. A language model may still generalize beyond the context if the instruction is loose. The notebook therefore includes explicit constraints: use the vault context, cite source filenames, and admit when the context does not contain enough material. These constraints are not merely stylistic. They are safeguards against hallucination, which is the production of fluent but unsupported or false content. In a research workflow, unsupported fluency can be dangerous because it may appear intellectually polished while concealing a weak evidentiary basis.

The notebook generates longer work section by section rather than asking for an entire paper in one request. This is a practical design choice with several advantages. First, each section can have a focused retrieval query. The introduction may need notes about motivation and problem framing, while a methods section may need notes about implementation details. Second, section-level generation keeps the prompt smaller and more manageable. Third, it allows the researcher to review and revise each part before moving on. Long-form generation often drifts when a model is asked to produce everything at once. Section-by-section generation reduces that drift by giving each part a clearer local purpose.

Grounded generation also changes the role of the researcher. The researcher is not replaced by the model; instead, the researcher becomes an editor, curator, and evaluator of machine-assisted drafts. The model can help turn scattered notes into coherent prose, but the researcher must decide whether the argument is correct, whether citations are adequate, whether the tone fits the intended audience, and whether the generated structure reflects the actual project. The best use of the notebook is therefore iterative. Retrieve, generate, inspect, revise the prompt, retrieve again, and refine the draft. The system supports thinking by giving the researcher material to react to.

There is a further benefit: generated output can expose weaknesses in the vault. If the model repeatedly says that the context is insufficient, this may indicate that the researcher has not written enough notes on the topic, has stored them in a folder excluded from indexing, or has used vague note titles and headings that make retrieval difficult. In this sense, the notebook provides feedback about the archive itself. It can show not only what the vault contains but also what it lacks. That feedback can guide future reading, note-taking, and project planning.

The generation stage is therefore not the final act of an automated pipeline but the beginning of a writing loop. The model's response should be treated as a draft grounded in retrieved material, not as a finished scholarly product. Its value lies in helping the researcher assemble, compare, and articulate ideas that already exist in the vault. When used carefully, this makes the notebook a disciplined writing assistant: fast enough to support exploration, constrained enough to remain connected to evidence, and transparent enough for the researcher to inspect the path from note to paragraph.

8 A Simple Case of Use, Concretely

The simplest use case does not require the notebook to write a full paper. Imagine a researcher whose vault contains a note called `Quantum Computing Basics.md` and another called `Tax-Loss Harvesting.md`. These notes were written months apart, perhaps for entirely different projects. The first explains qubits, superposition, computational complexity, and the possibility that future quantum systems may change how certain optimization problems are solved. The second explains investment losses, portfolio rebalancing, and the tax strategy of realizing losses to offset gains. The researcher has never linked these notes in Obsidian and may not remember them as related. One day, however, the researcher asks: “Have I ever written anything connecting quantum computing to tax strategy?”

A traditional keyword search would struggle with this question. If neither note contains both “quantum” and “tax,” the search may return nothing. Even if each note is found separately, the system may not explain why they might be related. Semantic retrieval gives a different kind of support. The query can retrieve the quantum computing note because it matches the technological and computational side of the question. It can also retrieve the tax-loss harvesting note because it matches the financial strategy side. The generation step can then place the two retrieved notes next to each other and draft a careful answer: the vault may not contain a direct connection, but it does contain material from which a speculative connection could be developed.

This example shows why a knowledge vault is more than a storage container. Researchers often generate insight by bringing previously separate ideas into contact. A vault may contain the raw material for such connections, but the relationships may remain hidden because the notes were written at different times, tagged differently, or placed in unrelated folders. A RAG workflow can help surface these latent relationships. It does not prove that the connection is valid, but it lowers the cost of noticing that a connection might be worth investigating.

The same pattern applies to more conventional academic work. A sociologist might ask whether notes on organizational routines connect to notes on laboratory instrumentation. A historian might ask whether a set of archival notes contains recurring references to infrastructure, delay, or bureaucratic classification. A philosopher might ask whether arguments about explanation in one folder relate to notes about causation in another. In each case, the retrieval system searches by meaning across the archive, and the generation system helps articulate the possible relationship.

The example also demonstrates a responsible boundary. The model should not claim that the researcher has definitely developed a theory connecting quantum computing and tax strategy unless the retrieved notes support that claim. A better answer would distinguish between direct evidence and possible synthesis. It might say that the vault contains one note on quantum computing and another on tax-loss harvesting, but no passage explicitly connects them. It might then propose several ways the researcher could explore a connection, such as optimization, risk modeling, or future financial technology. This kind of answer is useful precisely because it is honest about what the archive does and does not contain.

In practical terms, this simple use case can become a daily research habit. Instead of opening Obsidian and manually browsing folders, the researcher can ask exploratory questions: “Where have I discussed implementation failure?” “Which notes connect embodiment with measurement?” “Do I have material for a section on method?” “What evidence do I have for this claim?” The notebook can retrieve likely passages and help draft summaries. The researcher then returns to the original notes, checks the context, and decides what to keep. The value is not that the system thinks on behalf of the researcher. The value is that it makes the researcher’s own accumulated thinking easier to rediscover and recombine.

9 Research and Writing Implications

The notebook has implications beyond the immediate task of searching an Obsidian vault. It points toward a broader change in how researchers may work with personal archives. Many writing tools treat notes as static references: they are stored, tagged, linked, and manually retrieved. A RAG workflow treats notes as an active corpus that can participate in drafting. This does not eliminate the need for careful reading or interpretation, but it changes the speed and texture of early-stage writing. Instead of beginning with a blank page, the researcher can begin with a generated synthesis of relevant notes, then revise, challenge, expand, or reject it.

This workflow can be especially helpful during transitions between reading and writing. Researchers often spend weeks collecting notes before they know exactly how an argument will be organized. When drafting begins, they must translate that dispersed material into sections. The notebook supports this translation by retrieving passages aligned with a section prompt and arranging them into prose. For example, a prompt for an introduction may retrieve notes about the research problem, while a prompt for background may retrieve conceptual definitions and prior literature. The result is not a finished article, but it can provide a scaffold that helps the researcher see what material is available for each part of the paper.

The workflow also encourages better source awareness. Because the prompt asks the model to cite originating note filenames, the generated draft can preserve a visible trail back to the vault. This is useful for revision. If a paragraph seems promising, the researcher can open the cited note and examine the full context. If a paragraph seems unsupported, the researcher can see whether the problem comes from weak retrieval, insufficient notes, or overactive generation. Source awareness turns the draft into an inspectable object rather than an opaque output.

At the same time, the notebook highlights limits that researchers must take seriously. Semantic similarity is not the same as correctness. A retrieved passage may be topically related but methodologically inappropriate. A generated synthesis may sound coherent while flattening important distinctions. A filename citation is not the same as a formal bibliographic citation. Researchers using the notebook must therefore maintain a strong boundary between drafting support and scholarly validation. The system can help find and organize material, but the researcher remains responsible for claims, evidence, citations, and interpretation.

The notebook also suggests a new kind of literacy. Researchers increasingly need to understand not only how to write notes but how computational systems read notes. Headings, summaries, and consistent terminology become useful not only for human rereading but also for machine retrieval. A well-structured note with a clear title and focused paragraphs is easier for both the researcher and the embedding system to use. In this way, RAG can influence upstream writing habits. The researcher may begin to write notes with future retrieval in mind, adding concise summaries, explicit definitions, and clearer conceptual links.

Finally, the workflow can support intellectual humility. Because the system retrieves from an actual vault, it can reveal gaps as well as strengths. A researcher may discover that they have many notes gesturing toward a topic but few notes containing evidence. They may find repeated concepts but no clear definitions. They may see that an argument they thought was well developed exists only as scattered fragments. These discoveries are valuable. A good research assistant does not only confirm what the researcher hopes is present; it also helps identify what still needs to be read, clarified, or written. Used in this way, the notebook becomes part of a reflective research process rather than a shortcut around it.

10 Closing Thought

The notebook teaches a transferable pattern: index once, retrieve many times, and generate on demand. This pattern is useful because it separates preparation from use. Indexing organizes the vault into a searchable form. Retrieval selects the material most relevant to a particular

question. Generation turns that material into a readable draft. Once the researcher understands this sequence, the same design can be adapted to many domains. The corpus might be a personal Obsidian vault, a folder of interview transcripts, a collection of policy documents, a literature-review library, or a company's internal documentation. The technical details may change, but the conceptual structure remains stable.

The most important lesson is that RAG is not simply a way to make a language model smarter. It is a way to give the model a specific memory for a specific task. In this notebook, that memory is the researcher's own vault. This matters because the quality of the output depends not only on the model but also on the archive, the chunking strategy, the retrieval query, and the prompt instructions. A strong model with poor context may produce weak writing. A modest model with well-retrieved context may produce surprisingly useful drafts. The center of the workflow is therefore not the model alone, but the relationship between the model and the curated knowledge source.

The notebook also demonstrates that useful AI systems do not always need to be large or complicated. A small pipeline can teach the essential mechanics clearly. It can show where data enters, how text becomes vectors, how similarity search works, how retrieved passages are formatted, and how generation is constrained. This transparency is valuable for researchers who want to use AI critically rather than mystically. By seeing the pipeline, they can ask better questions about failure, bias, privacy, and evidence.

For academic writing, the promise of the workflow is not automatic authorship but assisted recall and synthesis. A vault-based RAG system helps the researcher rediscover what they have already written, notice relationships across distant notes, and begin drafting from grounded material. It can reduce the friction of starting a section and make the archive feel more alive. Yet it still requires human judgment at every important point. The researcher chooses the corpus, frames the query, reviews the retrieved chunks, edits the generated prose, and verifies the final claims.

That balance is the reason the notebook is a useful simple case. It neither rejects AI writing tools nor treats them as autonomous scholars. Instead, it places them inside a disciplined workflow where retrieval supplies grounding and the researcher supplies interpretation. The vault remains the intellectual base. The model becomes a means of reorganizing and articulating that base. When used with care, this arrangement can make personal knowledge systems more powerful without surrendering the standards of research writing. The final lesson is therefore practical and methodological: the future of AI-assisted scholarship should not be built on detached generation alone, but on transparent systems that connect writing back to the materials from which it claims to speak.

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 2020.
- [2] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data*, 2019.
- [3] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. "Dense Passage Retrieval for Open-Domain Question Answering."

Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020.